



Data Science with Python 3

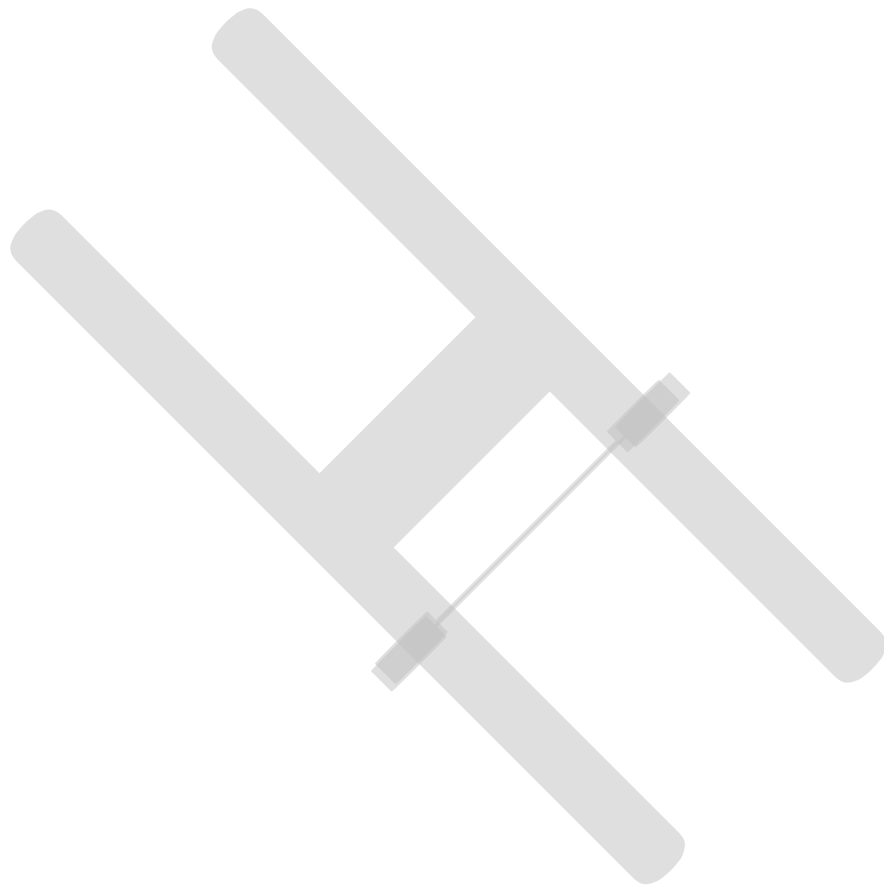


Prepared by:
Dr. Heba El Hadidi

2024-2025

Contents

Chapter Two-Data Acquisition and Processing	3
References	37



Chapter Two-Data Acquisition and Processing

Contents:

2.1 Introduction to Data Acquisition

2.2 Data Preprocessing and Techniques

2.2.1 Data Cleaning

2.2.2 Data Integration

2.2.3 Data Transformation

2.2.4 Data Reduction

2.2.4.1 Dimensionality Reduction

2.2.4.2 Numerosity Reduction

2.2.4.3 Data Compression

2.3 Data Mining

2.3.1 Data Mining Applications

2.4 Data Interpretation

2.5 Question

OBJECTIVES

1. Introduction to Data Acquisition
2. familiarize with the different types of data
3. Provide the concept of data pre-processing and familiarize with the various reprocessing techniques
4. Familiarize with the concept of data mining and data interpretation

2.1 INTRODUCTION TO DATA ACQUISITION

The process of gathering data and making it useful by filtering and cleaning it as per the business requirement is termed as **data acquisition** it is the most important step of data science, but the data acquired must be suitable to the problem in hand, else the output may be inaccurate.

It is very important to acquire up to date data.

The characteristics of big data namely **volume**, **velocity**, **variety**, and **value** are very important for the acquisition of data.

→ In data science, there is a *variety of data* that we need to deal with.

The **data** is majorly **characterized** as:

- **Structured Data:** The data which is available in some **stand format** is called structured data.

Structured data is **organized data**.

It is stored in **the row and column** structure like in a **relational database** and **excel sheets**. Some examples of structured data are names, numbers, geolocations, addresses etc.

- **Unstructured Data:** This data is **unorganized data**.

There is **no particular format** for unstructured data.

It is available in a **variety of formats** like texts, pictures, videos etc.

Unstructured data is **more difficult** to search and requires processing to become **understandable**.

- **Semi- Structured data:** This data is basically a **semi-organised** data.

It is generally a form of data that **do not conform** to the formal structure of data.

Log files are the examples of this type of data.

2.2 **DATA PREPROCESSING AND TECHNIQUES**

The data acquired need to be pre-processed to make it usable. The raw data may be

inconsistent, erroneous, incomplete and not in the required format. These issues need to be resolved to make the data usable. Data Preprocessing is a collaborative term used for the activities involved in transforming the real-world data or raw data into a usable form to make it more valuable and to get it in the required format.

The preprocessed data is cleaner and more valuable and hence used as final training set. Data preprocessing is a very essential step, as more clean and inconsistent data we get, better shall be the final output. In other words, data processing improves the quality of data. The huge size of data collected from heterogenous sources leads to anomalous data. Data preprocessing has become a vital and most fundamental step considering the fact that high quality data leads to better models and predictions.

Data preprocessing techniques are majorly categorized in the following methods:

- i. Data Cleaning
- ii. Data Integration
- iii. Data Transformation
- iv. Data Reduction

2.2.1 Data Cleaning

- معالجة البيانات المفقودة - **Processing missing data**
- اكتشاف القيم الشاذة **Detecting outliers**
- **Feature Engineering**

Data Cleaning involves removing duplicate data, filling missing values, identifying outliers, smoothening noise and remove data inconsistencies.

Why Is Data Cleaning Important?

- Improves model accuracy
- Reduces bias and errors
- Ensures consistency
- Makes analysis reliable
- Prepares data for Machine Learning algorithms

The Main steps for cleaning data:

◆ Handle missing data:

During analysis of data, it is important to understand why the missing values exist. Whether the missing value exist because they were not recorded or they imply that data does not exist for the missing values.

Missing data may be handled by using the following ways:

i) In case, the values were not recorded, it becomes essential to fill up the values by guessing based on the other values in that column and row. This is called **imputation**.

E.g. if a value is missing for gender, it is understood that the value was not recorded, so we need to analyse data and give it a value, we cannot leave the value blank in this case. Therefore, you can input missing values based on other observations; but there is a chance of losing integrity of the data because these values are being filled based on assumptions and not actual observations.

ii) If a value is missing because it doesn't exist e.g. the height of the oldest child of someone who doesn't have any children. In this case, it would make more sense to either leave it empty or to add a third value like NA or NaN. The NA or NaN values should be replaced with 0. Panda's fillna() function to fill in missing values in a data frame can be used.

iii) In case, it is not possible to figure out the reason for the missing, then that particular value can be dropped. Pandas function, `dropna()` can be used to do this, but doing this can drop or lose information, so be mindful of what is being removed or dropped this before removing it.

Replace with:

- Mean
- Median
- Mode

◆ Remove duplicate or irrelevant observations

It is very important to remove inconsistencies in dataset like removing unwanted observations including duplicate and irrelevant data entries.

Duplicate values may be caused at the time of data collection e.g. the names of the countries may have repeated valued like *NewZealand* and *New Zealand*; *Pakistan* and *pakistan*.

Inconsistencies in capitalizations and trailing white spaces are very common in text data. The data set can be cleaned using available function in Python or R. functions like `unique`, `sort`, `lower()`, `upper()`, `strip()` can be used to handle inconsistencies.

◆ **Fix structural errors:** Structural errors are when you measure or transfer data and notice strange naming conventions, typos, or incorrect capitalization. These inconsistencies can cause mislabelled categories or classes. For example, you may find —N/A|| and —Not Applicable|| both appear, but they should be analysed as the same category. Set the single date format in case there are multiple date formats in a single column.

- ◆ **Filter unwanted outliers** is essential to improve the performance of the data. If an outlier is found to be irrelevant it must be removed.

Detection methods:

- Boxplot
- Z-score
- **IQR (Interquartile Range)**

Example using IQR:

```
Q1 = df['salary'].quantile(0.25)
```

```
Q3 = df['salary'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df = df[(df['salary'] >= Q1 - 1.5*IQR) & (df['salary'] <= Q3 + 1.5*IQR)]
```

- ◆ **Standardizing Data Formats**

Examples:

- **Convert dates to consistent format**
- Standardize text values (e.g., "Male", "male", "M" → "Male")
- Convert strings to lowercase

```
df['gender'] = df['gender'].str.lower()
```

```
df['date'] = pd.to_datetime(df['date'])
```

- ◆ **Encoding Categorical Variables**

Machine Learning models require numerical data.

Techniques???

- ◆ **Noisy Data:**

Noisy data refers to data that **contains errors**, **random variations**, or **irrelevant information** that distort the true signal.

Random variance in the data is called noise. The following methods called **smoothing** techniques are used to handle noisy data:

- a) **Binning Method:** This method is also known as **discretization**, is used to smooth sorted data values by consulting the values around it i.e., the neighbouring values. It is a local smoothening method, since it refers to the neighbouring values.

This is a process of **converting continuous data into a set of data intervals.**

Continuous attribute values are substituted by small interval labels. This makes the data easier to study and analyze.

Data discretization can be classified into two types: **supervised discretization**, where the class information is used, and **unsupervised discretization**, which is based on which direction the process proceeds, i.e., 'top-down splitting strategy' or 'bottom-up merging strategy'.

For example, the values for the age attribute can be replaced by the interval labels such as (0-10, 11-20...) or (kid, youth, adult, senior).

In this method, the entire data is divided into equal segments called **bins** or **buckets**. Smoothing by binning is done by one of the following methods:

Smoothening by Bin Means: In smoothing by bin means, each value in a bin is replaced by the mean value of the bin.

- **Smoothening by Bin Median:** In smoothing by bin means, each value in a bin is replaced by the median value of the bin.

- **Smoothening by Bin Boundary:** In smoothing by bin boundaries, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.

Algorithm:

1. Sort the dataset.
2. Partition the dataset into n' segments. Each segment should contain approximately same number of data elements. Partitioning can be done using either of the following methods namely: equal width binning, equal frequency binning or entropy-based binning.
3. Calculate the arithmetic mean or media or replace by boundary (min and max value)
4. Replace each data element in each bin by the calculated mean/ median/ boundaries.

Example: Given data: 18, 22, 6, 6, 9, 14, 20, 21, 12, 18, 18, 16. Illustrate binning by mean, median and boundary replacement. Given bin depth=3

Step1: Sort the data: 6, 6, 9, 12, 14, 16, 18, 18, 18, 20, 21, 22,

Step 2: Partition the data into equal frequency bins of size of bin depth(n/d) where n = no. of elements and d = bin depth

$N/D = 12/3 = 4$ bins

$$\frac{12}{4} = 3$$

Bin 1: 6, 6, 9

Bin 2: 12, 14, 16

Bin 3: 18, 18, 18

Bin 4: 20, 21, 22

Step 3: Calculate Arithmetic mean

Arithmetic mean = Sum of observations ÷ number of observations

$$\text{Bin 1} = (6+6+9)/3 = 21/3 = 7$$

$$\text{Bin 2} = (12+14+16)/3 = 42/3 = 14$$

$$\text{Bin 3} = (18+18+18)/3 = 54/3 = 18$$

$$\text{Bin 4} = (20+21+22)/3 = 63/3 = 21$$

Step 4: Replace each data element in each bin by the calculated mean

Bin 1: 7, 7, 7

Bin 2: 14, 14, 14

Bin 3: 18, 18, 18

Bin 4: 21, 21, 21

- **Binning using Median:** In this method, Step 1 and step 2 are same.

Step 3: Calculate Median (50% percentile)

Median is the observation 2 in each bin

Step 4: Replace each data element in each bin by the calculated mean

Bin 1: 6, 6, 6 ✓

Bin 2: 14, 14, 14 ✓

Bin 3: 18, 18, 18 ✓

Bin 4: 21, 21, 21 ✓

- **Binning using Boundary Values:** In this, we keep the minimum as well as maximum values.

Bin 1: 6, 6, 9

Bin 2: 12, 12, 16

Bin 3: 18, 18, 18

Bin 4: 20, 20, 22

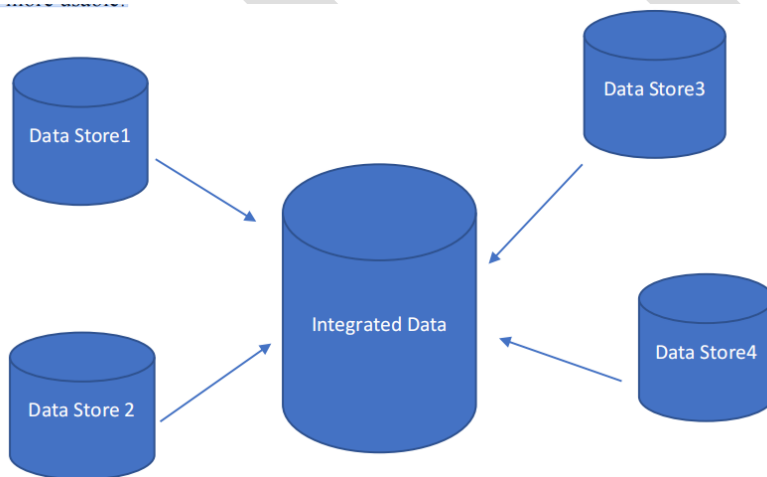
b) Regression: Regression is used to find a mathematical equation to fit the data to smooth out the noise. Regression may be linear or multiple. Linear regression is used to find the best line that fits two variables such that one predicts the other. Multiple linear regression involves more than two variables are involved.

c) Clustering: This approach is useful for organizing similar data in groups or clusters. The outliers may be detected by clustering. Data Integration makes data more

comprehensive and more usable.

2.2.2 Data Integration

Data Integration is a vital step in which the data acquired from multiple sources is integrated into a single centralized location. Data Integration makes data comprehensive and more usable.



The most common approaches to integrate data are:

- a) Data Consolidation ✓ ETL
- b) Data Propagation ✓
- c) Data Virtualization ✓
- d) Data Warehousing ✓

a) Data consolidation means to consolidate data from several separate sources into one data store, so that it is available to all the stake holders to enable better decision making.

It involves eliminating redundancies, removing inaccuracies before consolidating to a single store. The most common data consolidation techniques are ETL (Extract, Transform, Load), Data virtualization and Data warehousing.

- **ETL** is the most widely used data consolidation technique. The data is first extracted from multiple sources, then it is transformed into an understandable format by using various functions like sorting, aggregating, cleaning etc and then transfer it to a centralized store like another database or data ware house.

The ETL process cleans, filters, and transforms data, and then applies business rules before data populates the new source. ETL is further of two types namely Real time ETL used in real time systems and Batch processing ETL used for high volume databases.

- **Data Virtualization:** In this method, the data stays in the original location, but changes are made in a virtual manner and can be seen in a consolidated manner by the users. It is a logical layer that amalgamates data from various sources without performing actual ETL process. It is an abstraction such that only the required data is visible to the users without requiring technical details about the location or structure of the data source. It provides enhanced data security.

- **Data Warehousing** is the integration of data from multiple sources to a centralized source to facilitate decision making, reporting and query handling. A centralized source of data enables better decision making.

b) Data Propagation involves copying data from one location i.e., source to another location i.e., target location. It is event driven. These applications usually operate online and push data to the target location. They are majorly useful for real time data movement such as workload balancing, backup and recovery. Data propagation can be done asynchronously or synchronously.

The two methods for data propagation are: Enterprise Application Integration (EAI) and Enterprise Data Replication (EDR). The key advantage of data propagation is that it can be used for real-time / near-realtime data movement and can also be used for workload balancing, backup and recovery. EAI is used majorly for the exchange of messages and transactions in real-time business transaction processing; whereas for transfer of voluminous data between databases, is used.

c) Data Virtualization: In this, data is not stored in a single location, but is abstracted and can be viewed as unified view of data from multiple sources. Data Federation is a form of data virtualization, supported by Enterprise information technology (EII). EII products have evolved from two different technological backgrounds – relational DBMS

and XML, but the current trend of the industry is to support both approaches, via SQL (ODBC and JDBC) and XML (XML Query Language - XQuery - and XML Path Language - XPath) data interfaces.

d) Data Warehousing is the integration of data from multiple sources to a centralized source to facilitate decision making, reporting and query handling. A centralized source of data enables better decision making.

2.2.3 Data Transformation

After the data has been acquired, it is cleaned as discussed above. The clean data may not be in a standard format. The process of changing the structure and format of data to make it more usable is called data transformation. Data transformation may be constructive, destructive, aesthetic or structural.

- *Constructive Data Transformation* involves adding, copying, and replicating data.
- *Destructive Data Transformation* involves deleting fields and records.
- *Aesthetic Data Transformation* involves standardizing salutations or street names.
- *Structural Data Transformation* involves renaming, moving, and combining columns in a database.

Data Transformation may require smoothing, aggregation, discretization, attribute Construction, generalization and normalization to make data manageable.

Scripting languages like Python or domain-specific languages like SQL are usually used for data transformation.

Data Transformation Techniques

1. Data smoothing
2. Attribute construction
3. Data aggregation
4. Data normalization
5. Data discretization
6. Data generalization

2.2.4 Data Reduction

It is the process of reducing the volume of data such that data integrity is preserved. i.e., the volume of data is reduced but the results of data mining before and after mining are the same. Data reduction increasing the efficiency of data mining. It helps in reducing the storage requirement, reduced computation time and removal of redundancy. The various techniques used for data mining are:

Dimensionality reduction, numerosity reduction and data compression.

2.2.4.1 Dimensionality Reduction: is the transformation of data from high dimensional space to a low dimensional space. It is difficult to visualize data that has higher number of features. Sometimes, most of these features are correlated, and hence redundant. This is where the need of dimensionality reduction arises. In other words, dimensionality reduction is the process of reducing the number of random variables under consideration, by obtaining a set of principal variables. e.g., 3-dimensional data may be reduced to a 2D data. The various methods used for dimensionality reduction include:

- **Wavelet Transformation** is mostly used in image compression. It is a lossy method for dimensionality reduction, where a data vector X is transformed into another vector X' , such that both X and X' represent the same length. The result of wavelet transform can be truncated, unlike its original, thus achieving dimensionality reduction. Wavelet transforms are well suited for data cube, sparse data or data which is highly skewed.

- **Principal Component Analysis (PCA)** is applied to skewed and sparse data. In this method, the entire data set is represented by few independent tuples with 'n' attributes. This method was introduced by Karl Pearson. It works on a condition that while the data in a higher dimensional space is mapped to data in a lower dimension space, the variance of the data in the lower dimensional space should be maximum. It involves the following steps:

- i. Construct the covariance matrix of the data.

- ii. Compute the eigenvectors of this matrix.

iii. Eigenvectors corresponding to the largest eigenvalues are used to reconstruct a large fraction of variance of the original data.

Hence, lesser number of eigenvectors are left.

• **Attribute Subset Selection:** In this method, a subset of some selected attributes is created for reducing the volume of data. The goal of attribute subset selection is to find a minimum set of attributes such that dropping of those irrelevant attributes does not much affect the utility of data and the cost of data analysis could be reduced.

Attribute Subset Selection is done by the following methods:

1. Stepwise Forward Selection.
2. Stepwise Backward Elimination.
3. Combination of Forward Selection and Backward Elimination.
4. Decision Tree Induction.

2.2.4.2 Numerosity Reduction: is the reduction of original data and its representation in a smaller form. It can be done in two ways: parametric and non-parametric numerosity reduction.

i) Parametric Numerosity Reduction: In this method, only the data parameters are stored, instead of the entire original data. The data is represented using some model to estimate the data, so that only parameters of data are required to be stored, instead of actual data. Regression and Log-Linear methods are used for creating such models.

ii) Non- Parametric Numerosity Reduction methods are used for storing reduced representations of the data include *histograms, clustering, sampling* and *data cube aggregation*.

- **Histogram** is the data representation in terms of frequency. It uses binning to approximate data distribution and is a popular form of data reduction.
- **Clustering** divides the data into groups/clusters. This technique partitions the whole data into different clusters. In data reduction, the cluster representation of the data are used to replace the actual data. It also helps to detect outliers in data.
- **Sampling** can be used for data reduction because it allows a large data set to be represented by a much smaller random data sample (or subset).
- **Data Cube Aggregation** involves moving the data from detailed level to a fewer number of dimensions. The resulting data set is smaller in volume, without loss of information necessary for the analysis task.

2.2.4.3 Data Compression is a technique in which the original data is compressed for reduction. This compressed data can again be reconstructed to form the original data. If the data is reconstructed without losing any information, then it is a 'lossless' data reduction.

2.3 DATA MINING

Data mining is the process that helps in extracting information from a given data set to identify trends, patterns, and useful data. The objective of using data mining is to make

data-supported decisions from enormous data sets. Different types of data can be mined such as Data stored in database, data warehouse, transactional data and other types of data such as data streams, engineering design data, sequence data, graph data, spatial data, multimedia data, and more. In recent data mining projects, various major data mining techniques have been developed and used, including association, classification, clustering, prediction, sequential patterns, and regression.

- **Association** is a data mining technique useful to discover a link between two or more items. It uses if-then statements to show the probability of interactions between data items within large data sets. It finds a hidden pattern in the data set. It is commonly used to help sales correlations in data or medical data sets.
- **Clustering** is a data mining technique in which clusters are created of objects that share the same characteristics. Clusters relate to hidden patterns. It is based on unsupervised learning.
- **Classification** classifies items or variables in a data set into predefined groups or classes. It is based on unsupervised learning.
- **Prediction** is used in predicting the relationship that exists between independent and dependent variables as well as independent variables alone. It can be used to predict future trends. It analyses past events or instances in the right sequence to predict a future event.
- **Sequential patterns** is a data mining technique specialized for evaluating sequential data to discover sequential patterns. It comprises of finding interesting subsequence in

a set of sequences, where the stake of a sequence can be measured in terms of different criteria like length, occurrence frequency, etc.

2.3.1 Data Mining Applications

Data mining is useful in predictions, classification, clustering and trends, which makes it applicable to many domains like health care, fraud detection, education, market basket analysis, manufacturing, sales, customer relationship management, finance and banking etc.

2.4 DATA INTERPRETATION

The process of reviewing data through some predefined processes to be able to assign some meaning to the data and arrive at a relevant conclusion is called data interpretation. It involves taking the result of data analysis, making inferences on the relations studied, and using them to reach conclusions and develop recommendations. Data interpretation is done by analyst to make inferences from the data. There are two ways to interpret data namely **Qualitative** and **Quantitative**.

Qualitative Data Interpretation: Qualitative data also called categorical data, does not contain numbers, it consists of text, pictures, observations, symbols etc. The interpretation of patterns and themes in qualitative data is done using qualitative methods.

Quantitative Data Interpretation is done on quantitative data i.e. numerical data. It involves statistical methods such as mean, standard deviation, variance, frequency

distribution, regression etc. Data analysis and interpretation, regardless of method and qualitative/quantitative status, may include the following characteristics:

- Data identification and explanation
- Comparing and contrasting of data
- Identification of data outliers
- Future predictions

Data Interpretation is helpful in improving processes by identifying problems. Data interpretations is used for predicting trends after studying the patterns in the data. It is helpful in better decision making.

Preprocessing Examples

Take a look at the table below to see how preprocessing works. In this example, we have three variables: name, age, and company. In the first example we can tell that #2 and #3 have been assigned the incorrect companies.

Name	Age	Company
Karen Lynch	57	CVS Health
Elon Musk	49	Amazon
Jeff Bezos	57	Tesla
Tim Cook	60	Apple

We can use data cleaning to simply **remove these rows**, as we know the data was improperly entered or is otherwise corrupted.

Name	Age	Company
Karen Lynch	57	CVS Health
Tim Cook	60	Apple

Or, we can perform **data transformation**, in this case, manually, in order to fix the problem:

Name	Age	Company
Karen Lynch	57	CVS Health
Elon Musk	49	Tesla
Jeff Bezos	57	Amazon
Tim Cook	60	Apple

Once the issue is fixed, we can perform **data reduction**, in this case by descending age, to choose which age range we want to focus on:

Name	Age	Company
Tim Cook	60	Apple
Karen Lynch	57	CVS Health
Jeff Bezos	57	Amazon
Elon Musk	49	Tesla

2.5 QUESTIONS

1. What are the various types of data?
- 2.Explain the concept of data preprocessing. What are the techniques used for preprocessing of data?

3. Write a short note on Data mining.
4. What are the techniques used for data mining?
5. What is the importance of data Interpretation?

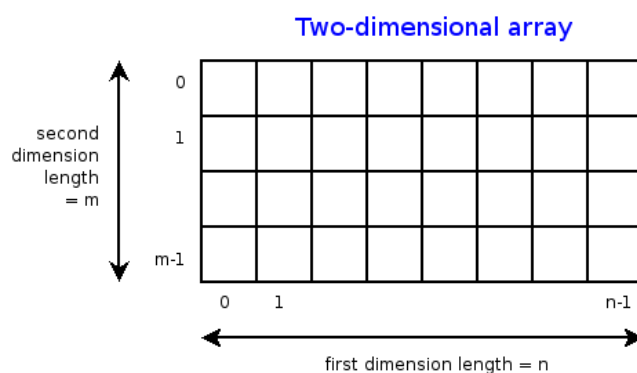
Data life cycle (continue)

1. **Data Representation**, i.e., what is the natural way to think about given data

One-dimensional Arrays, Vectors

0.1	2	3.2	6.5	3.4	4.1
-----	---	-----	-----	-----	-----

"data"	"representation"	"info"
--------	------------------	--------



Indexing

Slicing/subsetting

Filter

'map' → apply a function to every element

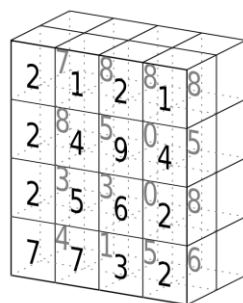
'reduce/aggregate' →

combine values to get a single scalar (e.g., sum, median)

Given two vectors: **Dot and cross products**

3	1	4	1
5	9	2	6
5	3	5	8
9	7	9	3
2	3	8	4
6	2	6	4

tensor of dimensions [6,4]
(matrix 6 by 4)



tensor of dimensions [4,4,2]

n-dimensional array operations

+

Linear Algebra

Matrix/tensor multiplication
Transpose
Matrix-vector multiplication
Matrix factorization

Sets: of Objects



Filter

Map

Union

Reduce/Aggregate

Given two sets, **Combine/Join** using “keys”

Group and then aggregate

Tables

company	division	sector	tryint
00nil_Combined_Company	00nil_Combined_Division	00nil_Combined_Sector	14625
apple	00nil_Combined_Division	00nil_Combined_Sector	10125
apple	hardware	00nil_Combined_Sector	4500
apple	hardware	business	1350
apple	hardware	consumer	3150
apple	software	00nil_Combined_Sector	5625
apple	software	business	4950
apple	software	consumer	675
microsoft	00nil_Combined_Division	00nil_Combined_Sector	4500
microsoft	hardware	00nil_Combined_Sector	1890
microsoft	hardware	business	855
microsoft	hardware	consumer	1035
microsoft	software	00nil_Combined_Sector	2610
microsoft	software	business	1215
microsoft	software	consumer	1395

Filter rows or columns

”Join” two or more relations

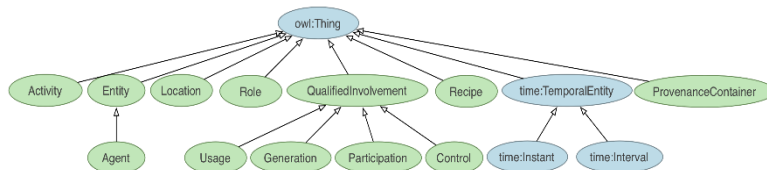
”Group” and “aggregate” them

Relational Algebra formalizes some of them

Structured Query Language (SQL)

Many other languages and constructs, that look very similar

Trees/Graphs



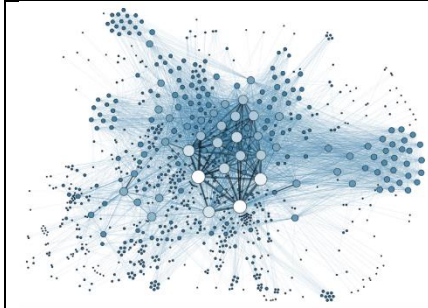
”Path” queries

Graph Algorithms and Transformations

Network Science

Somewhat more ad hoc and special-purpose

Changing in recent years



2. **Data Processing Operations**, which take one or more datasets as input and produce one or more datasets as output

Python Data Manipulation libraries:

1. **NumPy**: Python Library for Manipulating nD Arrays

Multidimensional Arrays, and a variety of operations including Linear Algebra

2. **Pandas**: Python Library for Manipulating Tabular Data

Series, Tables (also called DataFrames)

Many operations to manipulate and combine tables/series

3. **Relational Databases**

Tables/Relations, and SQL (similar to Pandas operations)

4. **Apache Spark**

Sets of objects or key-value pairs

MapReduce and SQL-like operations

Types of Data Analytics:

descriptive analytics, **diagnostic** analytics, **predictive** analytics, and **prescriptive** analytics. Each has its nature and the type of tasks performed in it.

NumPy DataTypes:

Variety of data types than are built-in in Python by default. Defined by the `numpy.dtype` class and include:

- `intc` and `intp` (used for indexing)
- `int8`, `int16`, `int32`, `int64`
- `float16`, `float32`, `float64`
- `complex64`, `complex128`
- `bool_`, `int_`, `float_`, `complex_`

Example:

```
import numpy as np
x=np.float32(1.0)
print(x) # 1.0
y=np.int_([1,2,4])
print(y) #array([1,2,4])
z=np.arange(3, dtype=np.uint8)
print(z)      # array([0,1,2], dtype=uint8)
print(z.dtype) #dtype('uint8')
```

Example:

```
import numpy as np
x=np.array([2,3,1,0])
x=np.array([ [1, 2.0], [ 0, 0], [1+ 1j, 3.]])
x=np.array([ [ 1.0+0.j, 2. + 0.j], [0. + 0.j, 0.+0.j], [1. +1.j, 3.+0.j]])
```

zeros(shape)- creates an array filled with **0** values with the specified shape. The default **dtype** is **float64**.

```
import numpy as np
ex=np.zeros((2,3)) #array([[0., 0., 0.], [0., 0., 0.]])
```

ones(shape)- creates an array filled with **1** values with the specified shape.

arange()- same as **range** in Python

```
import numpy as np
ex=np.arange(10) #array([0,1,2,3,4,5,6,7,8,9])
y=np.arange(2, 10, dtype=np.float) #array([2., 3., 4., 5., 6., 7., 8., 9.])
z=np.arange(2, 3, 0.2) #array([2., 2.2, 2.4, 2.6, 2.8])
```

linspace()- creates arrays with a specified number of elements and spaced equally between the specified beginning and end values.

```
import numpy as np
ex=np.linspace(1., 4., 6) #array([1., 1.6, 2.2, 2.8, 3.4, 4.])
```

random.random(shape)- creates arrays with random floats over the interval [0, 1)

```
import numpy as np
ex=np.random.random((2,3)) #array([ [0.75688597, 0.41759916, 0.35007419],
[0.77164187, 0.05869089, 0.98792864]])
```

Printing an array:

```
import numpy as np
a=np.arange(3)
print(a) #[0 1 2] or array([0,1,2])
b=np.arange(9).reshape(3,3)
print(b)
# [[ 0  1  2]
#   [ 3  4  5]
#   [ 6  7  8]]
c=np.arange(8).reshape(2,2,2)
print(c)
# [[ 0  1]
#   [ 2  3]]

# [[ 4  5]
```

[6 7]]

Indixing:

```
import numpy as np
a=np.arange(3)
x=np.arange(10)
print(x[2]) #2
print(x[-2]) #8
x.shape=(2,5) # now x is 2-dimensional
print(x[1,3]) #8
print(x[1,-1]) #9
#Sclicing

x=np.arange(10)

print(x[2:5]) #array([2,3,4])

print(x[:-7]) #array([0,1,2])

print(x[1:7:2]) #array([1,3,5])

y=np.arange(35).reshape(5,7)

print(y[1:5:2, : : 3]) #array([[7,10,13], [21,24,27]])
```

Array Operations

```
>>> a = np.arange(5)
>>> b = np.arange(5)
>>> a+b
array([0, 2, 4, 6, 8])
>>> a-b
array([0, 0, 0, 0, 0])
>>> a**2
array([ 0,  1,  4,  9, 16])
>>> a>3
array([False, False, False, False,  True], dtype=bool)
>>> 10*np.sin(a)
array([ 0.,  8.41470985,  9.09297427,  1.41120008, -
 7.56802495])
>>> a*b
array([ 0,  1,  4,  9, 16])
```

```
>>> a = np.zeros(4).reshape(2,2)
>>> a
array([[ 0.,  0.],
       [ 0.,  0.]])
>>> a[0,0] = 1
>>> a[1,1] = 1
>>> b = np.arange(4).reshape(2,2)
>>> b
array([[0, 1],
       [2, 3]])
>>> a*b
array([[ 0.,  0.],
       [ 0.,  3.]])
>>> np.dot(a,b)
array([[ 0.,  1.],
       [ 2.,  3.]])
```


There are also some built-in methods of `ndarray` objects.

Universal functions which may also be applied include `exp`, `sqrt`, `add`, `sin`, `cos`, etc.

```
>>> a = np.random.random((2,3))
>>> a
array([[ 0.68166391, 0.98943098,
        0.69361582],
       [ 0.78888081, 0.62197125,
        0.40517936]])
>>> a.sum()
4.1807421388722164
>>> a.min()
0.4051793610379143
>>> a.max(axis=0)
array([ 0.78888081, 0.98943098,
        0.69361582])
>>> a.min(axis=1)
array([ 0.68166391, 0.40517936])
```

ARRAY OPERATIONS

An array shape can be manipulated by a number of methods.

`resize(size)` will modify an array in place.

`reshape(size)` will return a copy of the array with a new shape.

```
>>> a =
np.floor(10*np.random.random((3,4)))
>>> print(a)
[[ 9. 8. 7. 9.]
 [ 7. 5. 9. 7.]
 [ 8. 2. 7. 5.]]
>>> a.shape
(3, 4)
>>> a.ravel()
array([ 9., 8., 7., 9., 7., 5., 9.,
        7., 8., 2., 7., 5.])
>>> a.shape = (6,2)
>>> print(a)
[[ 9. 8.]
 [ 7. 9.]
 [ 7. 5.]
 [ 9. 7.]
 [ 8. 2.]
 [ 7. 5.]]
>>> a.transpose()
array([[ 9., 7., 7., 9., 8., 7.],
       [ 8., 9., 5., 7., 2., 5.]])
```

Linear Algebra

One of the most common reasons for using the NumPy package is its linear algebra module.

```
>>> from numpy import *
>>> from numpy.linalg import *
>>> a = array([[1.0, 2.0],
               [3.0, 4.0]])

>>> print(a)
[[ 1.  2.]
 [ 3.  4.]]

>>> a.transpose()
array([[ 1.,  3.],
       [ 2.,  4.]])

>>> inv(a) # inverse
array([[ -2. ,  1. ],
       [  1.5, -0.5]])
```

```
>>> u = eye(2) # unit 2x2 matrix; "eye" represents "I"
>>> u
array([[ 1.,  0.],
       [ 0.,  1.]])
>>> j = array([[0.0, -1.0], [1.0, 0.0]])
>>> dot(j, j) # matrix product
array([[ -1.,  0.],
       [  0., -1.]])
>>> trace(u) # trace (sum of elements on diagonal)
2.0
>>> y = array([[5.], [7.]])
>>> solve(a, y) # solve linear matrix equation
array([[ -3.],
       [  4.]])
>>> eig(j) # get eigenvalues/eigenvectors of matrix
(array([ 0.+1.j,  0.-1.j]),
 array([[ 0.70710678+0.j,  0.70710678+0.j],
       [ 0.00000000-0.70710678j,
        0.00000000+0.70710678j]]))
```

SciPy

SciPy is a collection of mathematical algorithms and convenience functions built on the **NumPy** extension of Python. It adds significant power to the interactive Python session by providing the user with high-level commands and classes for manipulating and visualizing data.

Basically, SciPy contains various tools and functions for solving common problems in scientific computing.

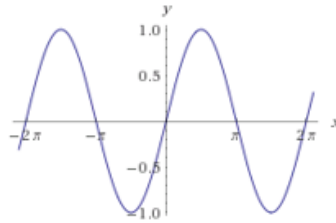
Some functionality:

- Special mathematical functions (scipy.special) -- elliptic, bessel, etc.
- Integration (scipy.integrate)
- Optimization (scipy.optimize)
- Interpolation (scipy.interpolate)
- Fourier Transforms (scipy.fftpack)
- Signal Processing (scipy.signal)
- Linear Algebra (scipy.linalg)
- Compressed Sparse Graph Routines (scipy.sparse.csgraph)
- Spatial data structures and algorithms (scipy.spatial)
- Statistics (scipy.stats)
- Multidimensional image processing (scipy.ndimage)
- Data IO (scipy.io) – overlaps with pandas, covers some other formats

SciPy Example: integration

Compute:

$$\int_a^b \sin x \, dx$$



scipy.integrate

np.sin function defines the sin function

to compute the definite integral from x:0 to pi using the quad function

```
>>> res = scipy.integrate.quad(np.sin, 0, np.pi)
>>> print(res)
(2.0, 2.220446049250313e-14) # 2 with a very small error
margin!
>>> res = scipy.integrate.quad(np.sin, -np.inf, +np.inf)
>>> print(res)
(0.0, 0.0) # Integral does not converge
```

```
>>> sample_x = np.linspace(0, np.pi, 1000)
>>> sample_y = np.sin(sample_x) # Creating 1,000 samples
>>> result = scipy.integrate.trapz(sample_y, sample_x)
>>> print(result)
1.99999835177

>>> sample_x = np.linspace(0, np.pi, 1000000)
>>> sample_y = np.sin(sample_x) # Creating 1,000,000
samples
>>> result = scipy.integrate.trapz(sample_y, sample_x)
>>> print(result)
2.0
```

References

- 1- Joel Grus, “Data Science from Scratch First Principals with Python”, 2nd edition, O’REILLY, USA, 2019.
- 2- Laura Lagual and Santi Segui, “**Introduction to Data Science** A Python Approach to Concepts, Techniques and Applications”, Springer 2017

